JC4003 Natural Language Processing

Lecture 4: N-gram Language Models

Assign a Probability to a Sentence

Dr Xiao Li xiao.li@abdn.ac.uk

Dr Ruizhe Li ruizhe.li@abdn.ac.uk

Dr Siwei Liu siwei.liu@abdn.ac.uk

Preface

The field of machine translation has undergone a significant evolution from rule-based approaches to statistical machine translation (SMT). Initially, systems relied heavily on intricate syntactic rules, but SMT introduced the use of statistical models that learn translation patterns from large corpora of bilingual text.

A pivotal technique in this paradigm shift was the N-gram language model, which estimates the fluency of translations by analysing the frequency of short word sequences. This approach bypasses the need for deep syntactic understanding, allowing translation systems to function effectively based on statistical regularities. This transition greatly enhanced the efficiency and practical application of machine translation systems.

Outline

What is a Language Model

N-gram Models

Smoothing

Evaluate Language Models (part 1)

Applications of N-gram Models

Language Model

Language Model

A language model is a computational system used to compute the probability of a sentence or sequence of words.

A sentence is composed of symbols (e.g., words), but not just any random arrangement of these symbols forms a sentence. Given any sequence of symbols, a language model can assign a probability indicating whether the sequence is a valid sentence in a particular language. In other words, it assesses whether the sentence is grammatically correct and coherent in that language.

A language model is a computational system → (English LM) → 99%

Een taalmodel is een computersysteem → (English LM) → 10%

Language Model

A language model is a computational system used to compute the probability of a sentence or sequence of words.

$$P(W) = P(w_1, w_2, \dots, w_n)$$

Its main purpose is to predict how likely a word or sequence of words is to appear in a given context. This is crucial for many tasks in NLP, such as translation, speech recognition, text generation, and spell checking.

Related task: probability of an upcoming word:

$$P(w_n | w_1, w_2, \dots, w_{n-1})$$

A model that computes either $P(W)$ or $P(w_n | w_1, w_2, \dots, w_{n-1})$ is called a language model.

Compute P(W)

How to compute the joint probability: its, water, is, so, transparent, that

$$P(\text{its water is so transparent that})$$

Recall the definition of conditional probabilities

$$P(B|A) = \frac{P(A,B)}{P(A)} \quad \text{i.e.} \quad P(A,B) = P(A)P(B|A)$$

More variables:

$$P(A,B,C,D) = P(A)P(B|A)P(C|A,B)P(D|A,B,C)$$

The Chain Rule in General:

$$P(w_1, w_2, \dots, w_n)$$
$$= P(w_1)P(w_2|w_1) \dots P(w_n|w_1, w_2, \dots, w_{n-1})$$
$$= \prod_i P(w_i|w_1, w_2, \dots, w_{i-1})$$

Compute P(W) (cont.)

How to compute the joint probability: its, water, is, so, transparent, that

$$P(\text{its water is so transparent that})$$
$$= P(\text{its}) \cdot P(\text{water}|\text{its}) \cdot P(\text{is}|\text{its water}) \cdot P(\text{so}|\text{its water is})$$
$$\cdot P(\text{transparent}|\text{its water is, so}) \cdot P(\text{that}|\text{its water is so transparent})$$

Word probability is estimated by word frequency, i.e. counting how frequently a word appears in a text corpus, divided by the total number of words in the corpus. E.g.:

$$P(\text{its}) = freq(\text{its}) = \frac{count(\text{its})}{count(all\ words)}$$

Similarly, for word combinations, the probability is based on the frequency of the entire sequence occurring together.

$$P(\text{its water}) = freq(\text{its water}) = \frac{count(\text{its water})}{count(all\ words)}$$

Compute P(W) (cont.)

The conditional probability $P(w_2|w_1)$ of a word w_2 given w_1 is the likelihood that w_2 follows w_1. It is calculated as

$$P(w_2|w_1) = \frac{P(w_1, w_2)}{P(w_1)}$$

where $P(w_2|w_1)$ is the joint probability of both words appearing together. E.g.:

$$P(\text{that}|\text{its water is so transparent}) = \frac{P(\text{its water is so transparen that})}{P(\text{itswater is so transparent})}$$

Problem: Too many combination probabilities! We'll never see enough data for estimating the long combinations.

Markov Assumption

Simplifying assumption: the probability of a word depends only on the preceding $n-1$ words, not on any earlier context.

$$P(w_i|w_1, w_2, \dots, w_{i-1}) \approx P(w_i|w_{i-k}, \dots, w_{i-2}, w_{i-1})$$

$$P(w_1, w_2, \dots, w_n) \approx \prod_i P(w_{i-k}, \dots, w_{i-2}, w_{i-1})$$

E.g.:

$$P(\text{that}|\text{its water is so transparent}) \approx P(\text{that}|\text{transparent})$$ ---- Unigram model

Or:

$$P(\text{that}|\text{its water is so transparent}) \approx P(\text{that}|\text{so transparent})$$ ---- Bigram model

N-gram Models

Like unigrams and bigrams, we can extend to trigrams, 4-grams, 5-grams, etc.

In general, this is an insufficient model of language because language has long-distance dependencies, e.g.: The computer which I had just put into the machine room on the fifth floor crashed.

But we can often get away with N-gram models.

Sentence Probability Calculation

Given a mini corpus, consisting of following sentences:

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green eggs and ham </s>
```

where <s> and </s> are two special symbols, denoting the start and end of sentences.

Here are the calculations for some of the bigram probabilities from this corpus:

$$P(\mathrm{I}|<\mathrm{s}>)=\frac{2}{3} \quad P(\mathrm{Sam}|<\mathrm{s}>)=\frac{1}{3} \quad P(\mathrm{am}|\mathrm{I})=\frac{2}{3} \quad P(</s>|\mathrm{Sam})=\frac{1}{2} \ \ldots$$

The probability of `<s> I am Sam </s>` :

$$P=P(\mathrm{I}|<\mathrm{s}>)P(\mathrm{am}|\mathrm{I})P(\mathrm{Sam}|\mathrm{am})P(</s>|\mathrm{Sam})=\frac{2}{3}\cdot\frac{2}{3}\cdot\frac{1}{2}\cdot\frac{1}{2}=\frac{1}{9}$$

Corpus

Corpora are a large and structured set of texts (nowadays usually electronically stored and processed).

Corpora are used to define a language or sub-language.

Corpora are used to do statistical analysis and hypothesis testing, checking occurrences or validating linguistic rules within a specific language territory.

Large Scale: Contains extensive text data, covering a wide range of linguistic phenomena.

Structured: Organised according to certain rules, making it easy to retrieve and analyse.

Representative: Texts come from diverse sources, aiming to reflect the real use of the target language.

Commonly Used Corpora

Text Classification
- AG_NEWS
- AmazonReviewFull
- AmazonReviewPolarity
- CoLA
- DBpedia
- IMDb
- MNLI
- MRPC
- QNLI
- QQP
- RTE
- SogouNews
- SST2
- STSB
- WNLI
- YahooAnswers
- YelpReviewFull
- YelpReviewPolarity

Language Modeling
- PennTreebank
- WikiText-2
- WikiText103

Machine Translation
- IWSLT2016
- IWSLT2017
- Multi30k

Sequence Tagging
- CoNLL2000Chunking
- UDPOS

Question Answer
- SQuAD 1.0
- SQuAD 2.0

Unsupervised Learning
- CC100
- EnWik9

Bigram Statistics From a Corpus

The table shows the bigram counts from a piece of a bigram grammar from the Berkeley Restaurant Project. Note that the majority of the values are zero.

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Bigram Estimates of Sentence Probabilities

P(<s> I want chinese food </s>) =

P(I|<s>) = 0.25

× P(want|I) = 0.33

× P(chinese|want) = 0.0011

× P(food|chinese) = 0.5

× P(</s>|food) = 0.68

= 0.000031

P(<s> I want foshan food </s>) =

P(I|<s>) = 0.25

× P(want|I) = 0.33

× P(foshan|want) = 0

× P(food|foshan) = 0

× P(</s>|food) = 0.68

= 0

Zero Probability Problem

Bigram Estimates of Sentence Probabilities

If a bigram's probability is 0, it causes the entire sentence probability to be 0. This becomes problematic when comparing the probabilities of multiple sentences; if they are all 0, comparison is impossible.

Due to the limitations of corpus size, 0 probabilities are inevitable, as it is difficult for the corpus to contain all possible word combinations. To address this issue, two approaches can be used:

- Choose a larger corpus
- Apply a smoothing algorithm to estimate a probability

P(<s> I want foshan food </s>) =

P(I|<s>) = 0.25

× P(want|I) = 0.33

× P(foshan|want) = 0

× P(food|foshan) = 0

× P(</s>|food) = 0.68

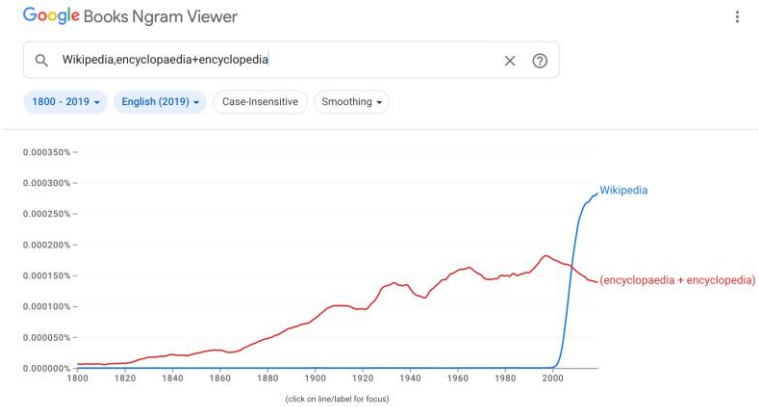
= 0

Google Books Ngram Viewer

The Google Books Ngram Viewer is an online search engine that charts the frequencies of any set of search strings using a yearly count of n-grams found in printed sources published between 1500 and now in Google's text corpora in English, Chinese (simplified), French, German, Hebrew, Italian, Russian, or Spanish.

The program can search for a word or a phrase, including misspellings or gibberish. The n-grams are matched with the text within the selected corpus, and if found in 40 or more books, are then displayed as a graph.

The dataset used by Google Books Ngram Viewer is also downloadable.



https://books.google.com/ngrams/

Smoothing

Add-one (Laplace) Smoothing

Add-one smoothing (also known as Laplace smoothing) is a technique used in probability and statistics to handle the problem of zero probabilities in categorical data. This is particularly useful in NLP for language modelling and other tasks involving n-gram models.

Add-one smoothing addresses the zero-probability problem by adding a small constant (usually 1) to all event counts. This ensures that every possible event has a non-zero probability i.e.

$$P(w_i) = \frac{count(w_i) + 1}{count(all\ words) + V} \qquad P(w_{i+1}|w_i) = \frac{count(w_i, w_{i+1}) + 1}{count(w_i) + V}$$

Where V is the size of the vocabulary.

Add-one (Laplace) Smoothing

Add-one smoothed bigram probabilities examples.

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

Adjusted Count

Using a smoothing algorithm, we can define the adjusted count c^*, which allows us to infer word counts from the smoothed probabilities and observe the actual impact of the smoothing algorithm on the corpus statistics.

$$P(w_i) = \frac{c_i}{N} \quad \Longrightarrow \quad c_i = P(w_i) \cdot N$$

where c_i and N denotes the word count of w_i and corpus word count.

By replacing $P(w_i)$ by the smoothed probability, it gives:

$$c_i^* = \frac{c_i + 1}{N + V} \cdot N$$

Adjusted Count

Limitation: the reduction may be very large!

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0						
spend	1	0						

Bigram word count

Bigram word count with smoothing

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

Add-one (Laplace) Smoothing

Advantages:

- Simplicity: Easy!
- Non-zero Probabilities: Ensures that no probability is zero, thus avoiding issues with unseen events.

Disadvantages:

- Over-smoothing: Can make the probabilities of rare events too high compared to more common events.
- Large reduction: If the context word is rare, the reduction can be very large.

Interpolation

Interpolation combines probabilities from multiple n-gram models (e.g., unigram, bigram, trigram) by weighting and summing them. Instead of completely falling back to a lower-order model, interpolation blends the probabilities from different levels, providing a more balanced estimate that leverages both specific and general patterns in the data.

E.g. when we use trigram, we use unigram, bigram, and trigram together, then, use three λs to balance the weights.

$$\hat{P}(w_3|w_1, w_2) = \lambda_1 P(w_3) + \lambda_2 P(w_3|w_2) + \lambda_3 P(w_3|w_1, w_2)$$

$$\lambda_1 + \lambda_2 + \lambda_3 = 1$$

Backoff

Backoff is a technique used in language modelling where, if an n-gram (e.g., trigram) has a zero count, the model "backs off" to a lower-order n-gram (e.g., bigram or unigram) to estimate the probability.

This approach ensures that even if specific n-grams are not present in the training data, the model can still provide a probability estimate using broader context.

$$\hat{P}(w_3|w_1,w_2) = \begin{cases} P(w_3|w_1,w_2) & if count(w_1,w_2,w_3) > 0 \\ \alpha \cdot P(w_3|w_2) & otherwise \end{cases}$$

There, the additional corpus is also required to find α.

Stupid Backoff

Brants et al. (2007) show that with very large language models a much simpler algorithm may be sufficient. The algorithm is called stupid backoff. Stupid backoff gives up the idea of trying to make the language model a true probability distribution. There is no discounting of the higher-order probabilities. If a higher-order n-gram has a zero count, we simply backoff to a lower order n-gram, weighed by a fixed (context-independent) weight.

$$\hat{P}(w_i|w_{i-N+1:i-1}) = \begin{cases} P(w_i|w_{i-N+1:i-1}) & if\ count(w_{i-N+1:i-1}) > 0 \\ \alpha \cdot P(w_i|w_{i-N+2:i-1}) & otherwise \end{cases}$$

Brants et al. (2007) found that a value of $\alpha = 0.4$ worked well if the corpus is very large e.g. you use Google NGrams.

Evaluate Language Models (part 1)

1495 UNIVERSITY OF ABERDEEN

Extrinsic (in-vivo) Evaluation

To compare models A and B

1. Put each model in a real task
    - Machine Translation, speech recognition, etc.
2. Run the task, get a score for A and for B
    - How many words translated correctly
    - How many words transcribed correctly
3. Compare accuracy for A and B

Extrinsic evaluation not always possible

- Expensive, time-consuming
- Doesn't always generalize to other applications

Intrinsic (in-vitro) Evaluation

Intrinsic evaluation: perplexity

- Directly measures language model performance at predicting words.
- Doesn't necessarily correspond with real application performance
- But gives us a single general metric for language models
- Useful for large language models (LLMs) as well as n-grams

Training Sets & Test Sets

We train parameters of our model on a training set.

We test the model's performance on data we haven't seen.

- A test set is an unseen dataset; different from training set.
- Intuition: we want to measure generalization to unseen data
- An evaluation metric (like perplexity) tells us how well our model does on the test set.

Choosing Training & Test Sets

If we're building an LM for a specific task

- The test set should reflect the task language we want to use the model for

If we're building a general-purpose model

- We'll need lots of different kinds of training data
- We don't want the training set or the test set to be just from one domain or author or language.

Training on The Test Set

We can't allow test sentences into the training set

- Or else the LM will assign that sentence an artificially high probability when we see it in the test set
- And hence assign the whole test set a falsely high probability.
- Making the LM look better than it really is

This is called "Training on the test set"

Bad science!

Dev Sets

If we test on the test set many times we might implicitly tune to its characteristics

- Noticing which changes make the model better.

So we run on the test set only once, or a few times

That means we need a third dataset:

- A development test set or, devset.
- We test our LM on the devset until the very end
- And then test our LM on the test set once

Applications of N-gram Models

1495 UNIVERSITY OF ABERDEEN

Applications of N-gram Models

N-gram models have widespread applications in the field of NLP:

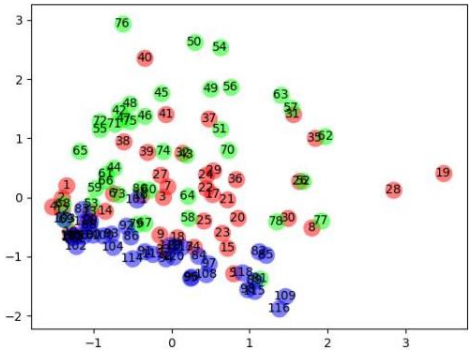
- Text classification
- Speech recognition
- Information retrieval
- …

Text Classification

Text classification using n-grams is straightforward. The typical approach involves converting each text into n-gram term frequencies, then analysing their similarities. This is often done by treating the n-gram term frequencies of each text as a vector and comparing these vectors.

Text Classification

A classic case is the proof that the last 40 chapters of 《红楼梦》(*Dream of the Red Chamber*) were not written by 曹雪芹 (Cao Xueqin). Researchers converted the function words in each chapter into vectors and compared them. They found significant differences in the distribution of vectors between the first 80 chapters and the last 40 chapters.



Statistics of words used in 《红楼梦》
Chapter 1-40
Chapter 41-80
Chapter 81-120

The same method was used to prove that *Henry VIII* was co-authored by William Shakespeare and John Fletcher, with each contributing approximately 50% of the work.

Conclusion

N-gram Language Models: Discussed how N-gram models estimate sentence probabilities by analyzing short word sequences and their frequencies.

Markov Assumption: Introduced the simplification that a word's probability depends only on the previous few words, reducing complexity.

Smoothing Techniques: Covered methods like Laplace smoothing to handle the zero-probability problem for unseen word combinations.

Applications: Highlighted the extensive use of N-gram models in tasks such as text classification, speech recognition, and information retrieval.

Limitations: Acknowledged that while N-gram models work well for short dependencies, they struggle with long-distance dependencies in language.